

# Coding for Artificial Intelligence

Week 5: Structured Data Representation

---

## Problem Set 5.1

Nested Dictionaries, Sets, and Immutability

*Lab Project: The Multi-Agent Intelligence Hub (MAIH)*

---

|                    |                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------|
| <b>Course</b>      | Coding for AI (Week 5)                                                                    |
| <b>Author</b>      | Dr. Prateek, Assistant Professor, School of AI                                            |
| <b>Format</b>      | In-class project, <b>1 hour 30 minutes</b> total,<br>inclusive of write-up and submission |
| <b>Total Marks</b> | 100                                                                                       |

Read this document in full before you begin writing code.

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Objective</b>                                        | <b>2</b> |
| <b>2</b> | <b>The Problem: Multi-Agent Intelligence Hub (MAIH)</b> | <b>2</b> |
| <b>3</b> | <b>Technical Requirements</b>                           | <b>2</b> |
| 3.1      | Nested State Registry . . . . .                         | 2        |
| 3.2      | Knowledge Sync Logic . . . . .                          | 3        |
| 3.3      | Conflict Resolution and the Immutability Test . . . . . | 3        |
| 3.4      | Efficiency Constraint: Summary Report . . . . .         | 3        |
| <b>4</b> | <b>Trace</b>                                            | <b>4</b> |
| <b>5</b> | <b>Edge Cases to Handle</b>                             | <b>4</b> |
| <b>6</b> | <b>Suggested Time Allocation</b>                        | <b>5</b> |
| <b>7</b> | <b>Deliverables</b>                                     | <b>5</b> |
|          | <b>Academic Integrity</b>                               | <b>5</b> |

## 1 Objective

So far you have worked with lists (ordered, mutable sequences) and, in Week 4–5, with loops that process them. This project introduces three new data structures that are central to representing structured, real-world state: **nested dictionaries**, **sets**, and **tuples**.

By the end of this project, you should be able to:

- (i) design and query a **dictionary of dictionaries** to represent multiple entities, each with several named attributes,
- (ii) use **set operations** (specifically intersection) to find information shared across several collections, and
- (iii) explain *why* tuples are immutable, and handle the `TypeError` that results from trying to mutate one directly.

*Time budget.* This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 6.

## 2 The Problem: Multi-Agent Intelligence Hub (MAIH)

In a simulated multi-agent AI environment, several autonomous agents each maintain their own position and their own accumulated knowledge. Your task is to build a small **global state registry** that tracks all agents at once, lets you find knowledge common to every agent, and correctly handles an agent changing position.

## 3 Technical Requirements

Your script, `intelligence_hub.py`, must implement the following.

### 3.1 Nested State Registry

Create a dictionary of dictionaries, `agent_registry`, keyed by `AgentID`. Each value is itself a dictionary with exactly two keys:

- `"Location"`: a tuple `(x, y)` of fixed integer coordinates.
- `"Knowledge"`: a **set** of strings, each string an “Information Bit” the agent has learned.

```
1 agent_registry = {
2     "Agent_Alpha": {
3         "Location": (10, 20),
4         "Knowledge": {"grid_map", "comm_protocol", "telemetry_sync"},
5     },
6     "Agent_Beta": {
7         "Location": (15, 5),
8         "Knowledge": {"grid_map", "comm_protocol", "power_grid"},
9     },
10    "Agent_Gamma": {
11        "Location": (0, 0),
12        "Knowledge": {"grid_map", "comm_protocol", "power_grid",
13                     "telemetry_sync"},
14    },
15 }
```

*Note on printing sets.* Python does **not** guarantee the order in which elements of a `set` are printed or iterated. If your console output lists the same knowledge bits as this document but in a different order, your program is still correct — only the *presence* of elements matters for a set, never their order.

### 3.2 Knowledge Sync Logic

Implement a function, `find_common_intelligence(registry)`, that returns the set of Information Bits known to **every** agent, using set intersection.

```
1 def find_common_intelligence(registry):
2     knowledge_sets = [
3         agent_data["Knowledge"] for agent_data in registry.values()
4     ]
5     return set.intersection(*knowledge_sets)
```

*Why this generalizes.* `set.intersection()` accepts any number of sets when called with the unpacking operator `*`. Writing the function this way means it works correctly whether `agent_registry` holds three agents or thirty, with no change to the code — unlike writing `alpha_knowledge & beta_knowledge & gamma_knowledge` by hand, which would need to be rewritten every time an agent is added or removed.

### 3.3 Conflict Resolution and the Immutability Test

These two requirements are related, and your script should implement them together in a single `relocate_agent(registry, log, agent_id, new_location)` routine.

1. **Immutability demonstration.** First, attempt to change the agent's existing `Location` tuple *in place* — e.g., `registry[agent_id]["Location"][0] = ...` — inside a `try` block, and catch the resulting `TypeError`. Print a message explaining what happened.

*Why this fails.* A tuple does not support item assignment: once created, the object at each of its positions is fixed. This is precisely what makes tuples suitable for representing something like a coordinate pair — you can rely on it never changing behind your back. It also means there is no way to “edit” a tuple; the only way to change an agent's location is to build a **new** tuple and reassign it.

2. **Correct relocation.** Reassign the whole "Location" entry to the new tuple: `registry[agent_id]["Location"] = new_location`.
3. **Movement logging with `setdefault`.** Record the move in a separate dictionary, `movement_log`, keyed by `AgentID`, where each value is a list of every location the agent has moved to. Use `setdefault` so that the first move for a given agent correctly creates a new empty list before appending to it, without an explicit `if agent_id not in movement_log` check:

```
1 log.setdefault(agent_id, []).append(new_location)
```

*Why `setdefault` matters here.* On an agent's *first* move, `agent_id` is not yet a key in `movement_log`. `setdefault(agent_id, [])` inserts an empty list under that key *only if it is not already present*, then returns the list either way — so the same line of code works correctly whether this is the agent's first move or its fifth.

### 3.4 Efficiency Constraint: Summary Report

Generate a `summary_report` dictionary mapping each `AgentID` to the number of knowledge bits it currently holds, using a **dictionary comprehension** (not a `for` loop with manual key

assignment):

```

1 summary_report = {
2     agent_id: len(data["Knowledge"])
3     for agent_id, data in agent_registry.items()
4 }
```

## 4 Trace

Running a correct implementation — printing the registry, the common intelligence, two relocations of `Agent_Alpha`, and the summary report — produces output equivalent to the following (element order within each printed set may differ on your machine; see the note in Section 3.1):

```

1 --- Common Intelligence Across All Agents ---
2 {'grid_map', 'comm_protocol'}
3
4 --- Relocating Agent_Alpha ---
5 Attempting direct mutation of Agent_Alpha's Location tuple...
6 TypeError caught: 'tuple' object does not support item assignment
7 Tuples are immutable; reassigning a new tuple instead.
8
9 Agent_Alpha new state: {'Location': (12, 22), 'Knowledge': {...}}
10 Movement Log: {'Agent_Alpha': [(12, 22)]}
11
12 --- Relocating Agent_Alpha again ---
13 Movement Log: {'Agent_Alpha': [(12, 22), (14, 25)]}
14
15 --- Summary Report (Dictionary Comprehension) ---
16 {'Agent_Alpha': 3, 'Agent_Beta': 3, 'Agent_Gamma': 4}
```

*How to check your own logic.* `Agent_Alpha` knows `grid_map`, `comm_protocol`, and `telemetry_sync`; `Agent_Beta` knows `grid_map`, `comm_protocol`, and `power_grid`; `Agent_Gamma` knows all four bits. The only bits present in **all three** sets are `grid_map` and `comm_protocol` — `telemetry_sync` is missing from `Agent_Beta`, and `power_grid` is missing from `Agent_Alpha`, so neither qualifies as common intelligence. Note also that the second call to `relocate_agent` **appends** to the existing list in `movement_log` rather than overwriting it — this is the behaviour `setdefault` is specifically designed to produce.

## 5 Edge Cases to Handle

1. **An agent with an empty Knowledge set.** Confirm that `find_common_intelligence` correctly returns an empty set in this case, since nothing can be common to a set that itself contains nothing.
2. **Relocating an agent that has never moved before.** Confirm your `setdefault` call creates the list correctly on the very first call, with no separate initialisation code required elsewhere in your script.
3. **Querying the summary report before any agent has learned anything** (an agent registered with `"Knowledge": set()`). Confirm your dictionary comprehension reports 0 for that agent rather than raising an error.

## 6 Suggested Time Allocation

| Phase                      | Description                                                 | Duration          |
|----------------------------|-------------------------------------------------------------|-------------------|
| Implementation             | Registry, knowledge sync, relocation, summary report        | 55 minutes        |
| Testing                    | Edge cases from Section 5; confirm output against Section 4 | 20 minutes        |
| Documentation & Submission | Header comment, final write-up                              | 15 minutes        |
| <b>Total</b>               |                                                             | <b>90 minutes</b> |

## 7 Deliverables

1. **Source Code:** A single file named `intelligence_hub.py` that prints the common knowledge bits between agents and a summary of agent states.
2. **Header Documentation:** A comment block at the top of your file explaining, in your own words, why `setdefault` is the right tool for the Movement Log, and why a tuple is the right choice (rather than a list) for representing `Location`.
3. **Submission Note:** A short paragraph (100–150 words) explaining what the `TypeError` in Section 3.3 tells you about how tuples are stored in memory, compared to lists.

## Academic Integrity

You may discuss approaches with peers during the supervised project session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute's academic integrity policy.